PYTHON MASTERY COURSE

Concepts, Code, and Practice

A plain-language, chapter-by-chapter guide from Python basics to AI/ML engineering.

Course progress: 3 of 37 chapters authored. This PDF contains the chapters currently available; it will grow as the remaining chapter files are added.

AVAILABLE CHAPTERS

- Chapter 01 - Introduction to Python
- Chapter 02 - Variables, Constants and Memory
- Chapter 03 - Python Data Types

Chapter 01 - Introduction to Python

Level: Beginner **Suggested study time:** 3-5 hours **Prerequisites:** None

This chapter introduces Python, its history, uses, tools, execution model, and basic input/output. Historical and comparison topics use examples rather than algorithms; step-by-step algorithms and dry runs are included wherever code performs a task.

Learning outcomes

By the end of this chapter, you should be able to:

- Explain what Python is and name its main strengths and tradeoffs.
- Describe the difference between source code, bytecode, and a Python virtual machine.
- Distinguish an interpreter, an interactive shell, an editor, an IDE, and a notebook.
- Run a Python file from a terminal and recognize the steps Python takes to execute it.
- Write readable code with comments, docstrings, correct indentation, `print()`, and `input()`.
- Trace a small program by hand and explain each value as it changes.

1. What is Python?

Definition

Python is a high-level, general-purpose programming language. A Python program is a set of instructions written in Python syntax. A Python implementation reads those instructions and performs the requested work.

- **High-level** means that most programs can be written without directly managing processor registers or memory addresses.
- **General-purpose** means Python can be used for many kinds of work, not only one specialized task.
- **Programming language** means it has rules for expressing instructions and combining them into programs.

Easy explanation

Think of a program as a recipe and the computer as a very literal cook. The recipe must be written in a language the computer's Python implementation can process. Python lets the recipe be relatively close to the way people describe steps, while still being precise enough for the computer.

PYTHON EXAMPLE

```
print("Hello, Python!")
```

`print` asks Python to display the text in parentheses. The quotes mark the text as a string; they are not displayed.

Dry run

Step	Instruction or value	What happens
1	`print("Hello, Python!")`	Python evaluates the string value.
2	`print(...)`	The text is sent to standard output, usually the terminal.

When Python is a good fit

Python is often a good choice when readability, fast development, automation, data work, or a large library ecosystem matters. It may not be the best choice for every hard real-time system, tiny microcontroller, or performance-critical inner loop. Those cases may use a different language, a native library, or a combination of languages.

2. History and features of Python

Short history

- Guido van Rossum began designing Python in the late 1980s.
- Python was first publicly released in 1991.
- Python 2.0 appeared in 2000.
- Python 3.0 appeared in 2008 and introduced changes that could not all remain compatible with Python 2.
- Python 2 reached end of life in 2020. New learning and projects should use a currently supported Python 3 release.

Python continues to evolve through community proposals, releases, and maintenance work. You do not need to memorize release dates to write programs; the important lesson is that the language and its tools change over time.

Important features

Feature	Meaning	Small example or consequence
Readable syntax	Many common operations use concise, familiar words and punctuation.	`total = price * count`
High-level	The language hides many low-level machine details.	You can work with a list without allocating each memory slot yourself.
Dynamically typed	The type belongs to a value; a variable name can later refer to a value of another type.	`item = 3`, then `item = "three"`
Multi-paradigm	The language supports procedural, object-oriented, and functional styles.	A script can use functions and classes together.
Automatic memory management	The implementation manages object memory for ordinary programs.	You normally do not manually free each string or list.
Standard library	Python ships with modules for common tasks.	`json`, `pathlib`, `datetime`, and `csv`
Broad ecosystem	Third-party packages add tools for many domains.	Data science, web development, testing, and automation
Portable	The same source can often run on different operating systems when dependencies and environment differences are handled.	A small script can usually run on Windows, macOS, and Linux.

The details of execution depend on the Python implementation. CPython is the most commonly used implementation, but it is not the only one.

3. Why Python is popular

Python is popular because it combines a relatively gentle learning curve with tools used in professional work.

1. **Readable code:** Python often needs fewer punctuation marks and less boilerplate than some alternatives.
2. **Fast experimentation:** An interactive shell and notebooks make it convenient to test a small idea.
3. **Useful libraries:** A library can provide a tested solution for a task that would otherwise take substantial work.
4. **Large community:** Tutorials, examples, forums, and maintained packages are widely available.
5. **Many application areas:** Skills from basic Python transfer to automation, data analysis, web applications, and machine learning.
6. **Good teaching language:** Beginners can focus on the problem before learning many low-level details.

Popularity does not mean Python is always the fastest language or the best tool for every job. Choose a language based on the problem, constraints, team, and available libraries.

Example: reuse a library

Python's standard library can parse JSON without writing a JSON parser yourself:

PYTHON EXAMPLE

```
import json

text = '{"course": "Python", "weeks": 8}'
course_info = json.loads(text)
print(course_info["course"])
```

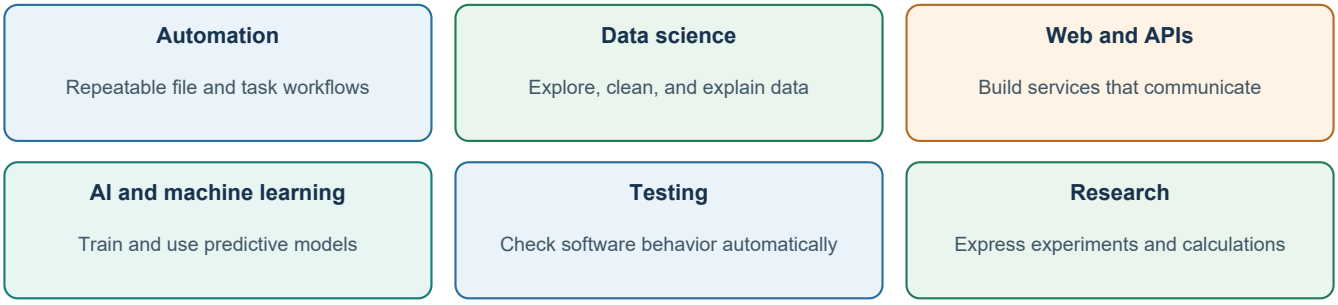
The `json` module handles the format rules; the program focuses on using the data.

4. Applications of Python

Python is used in many fields:

- **Automation:** Rename files, generate reports, or perform repeated checks.
- **Data science:** Clean data, calculate summaries, and create visualizations.
- **Machine learning and AI:** Prepare data, train models, and connect models to applications.
- **Web development:** Build server-side applications and APIs.
- **Testing and developer tools:** Test software and automate development tasks.
- **Education and research:** Express experiments and calculations clearly.
- **Networking and system administration:** Communicate with services and automate supported operations.

WHERE PYTHON IS USED



Example: a simple data workflow

A common data-science process can be described without committing to a particular library:

1. Read a dataset.
2. Inspect its columns and values.
3. Handle missing or invalid values.
4. Calculate summaries or train a model.
5. Check the result.
6. Save or present the result.

PYTHON EXAMPLE

```
scores = [72, 81, 90]
average = sum(scores) / len(scores)
print(f"Average score: {average}")
```

Dry run

Step	scores	Calculation	Result
1	`[72, 81, 90]`	`sum(scores)`	`243`
2	`[72, 81, 90]`	`len(scores)`	`3`
3	`[72, 81, 90]`	`243 / 3`	`81.0`
4	-	formatted output	`Average score: 81.0`

This tiny example illustrates the idea of transforming input data into a useful result. Real datasets require additional checks, which are covered in later chapters.

5. Python vs Java vs C++

These languages are all widely used, but they make different tradeoffs. The table describes common implementations and typical use, not an absolute rule.

Topic	Python	Java	C++
Common style	Concise, dynamically typed	Class-oriented, statically typed	Multi-paradigm, statically typed
Typical execution	Commonly compiled to bytecode and executed by a Python virtual machine	Compiled to JVM bytecode and executed by a Java virtual machine, often with JIT compilation	Commonly compiled to native machine code
Memory	Automatic memory management in common implementations	Automatic memory management through garbage collection	Manual control is possible; smart pointers and libraries can manage ownership
Common strengths	Scripting, automation, data science, fast development	Large services, enterprise software, Android-related ecosystems	Systems software, game engines, performance-sensitive components
Tradeoffs	Often slower for CPU-heavy pure-Python loops	More declarations and build setup than a tiny Python script	More complexity and responsibility for low-level details

Important comparison rules

- No language is automatically best for every project.
- Actual performance depends on the program, implementation, libraries, and workload.
- Python libraries may call optimized C, C++, or Fortran code, so a Python application can still use fast native components.
- Learn the concepts behind types, memory, algorithms, and testing; those concepts transfer between languages.

6. Compiled vs interpreted languages

Definitions

- A **compiler** translates a program from one representation to another before or during execution.
- An **interpreter** executes program instructions by reading and processing them at runtime.
- A **runtime** is the system that supports a running program, such as loading code and providing built-in operations.

These words describe ways of implementing and running a language. They are not always mutually exclusive categories. A language implementation can compile some form of code and also use an interpreter or just-in-time compiler.

Simple analogy

- **Ahead-of-time compilation:** Translate a whole instruction sheet before the task starts.
- **Interpretation:** Read an instruction, then perform it as you go.
- **Just-in-time compilation:** Observe frequently used work while running and compile selected parts for faster execution.

Common paths

Approach	Simplified flow
Native compiled program	Source code -> compiler -> machine code -> processor
Common CPython execution	Python source -> bytecode -> CPython virtual machine
JIT-enabled runtime	Source or bytecode -> runtime -> selected code compiled while running

The diagram is simplified: real implementations may include parsing, optimization, module loading, caching, and other steps.

Common misconception: "Python is interpreted" is a useful beginner-level shorthand, but it hides the fact that CPython normally compiles source to bytecode before its virtual machine executes it.

7. How Python works internally

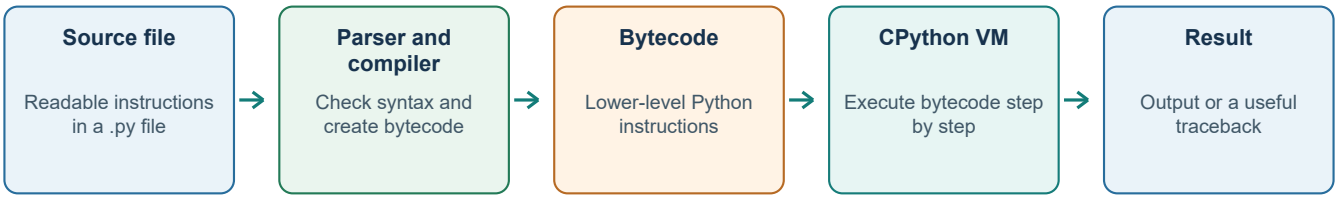
The basic idea

When a Python program runs, the implementation must turn the text in the source file into actions. In CPython, a simplified path is:

1. Read the source file.
2. Check that its syntax is valid and build an internal representation.
3. Compile the representation to Python bytecode.
4. Have the CPython virtual machine execute the bytecode.

5. Call built-in or library operations as the program needs them.

HOW A PYTHON SCRIPT RUNS



For imported modules, CPython may cache bytecode in a `__pycache__` directory. A cache is an optimization, not the original source and not something you normally edit by hand.

Example

PYTHON EXAMPLE

```
message = "Hello"
print(message)
```

Before the output appears, Python has to recognize a name assignment, a string literal, a function call, and the argument passed to that call. The virtual machine then performs the actions in order.

Practical way to inspect execution

Python's `dis` module can display bytecode for learning:

PYTHON EXAMPLE

```
import dis

def add(a, b):
    return a + b

dis.dis(add)
```

The exact bytecode and display can change between Python versions. Use this to understand that a translation step exists, not to memorize opcode names.

8. Python installation and setup

Windows setup

1. Download a supported Python 3 installer from the official Python website.
2. Run the installer. If the installer offers an **Add Python to PATH** option, enable it when appropriate for your setup.
3. Complete the installation.
4. Open a new PowerShell or Command Prompt window.
5. Check that the Python launcher works:

TERMINAL COMMAND

```
py --version
py -m pip --version
```

The version output should identify an installed Python 3 release. If `py` is not recognized, reopen the terminal first; if it still fails, review the installation and PATH options.

Create and run a test file

Save this as `hello.py`:

PYTHON EXAMPLE

```
print("Python is ready")
```

In the terminal, change to the directory that contains the file and run:

TERMINAL COMMAND

```
py hello.py
```

Expected output:

SAMPLE OUTPUT

```
Python is ready
```

Setup habits

- Use a supported Python 3 version.
- Use a project-specific virtual environment when a project has package dependencies; environments are covered in Chapter 22.
- Use `py -m pip ...` on Windows so `pip` is associated with the selected Python launcher.
- Do not install packages just because a tutorial lists them. Install only the dependencies the project needs.
- Keep project files in a folder you can find, and use clear names such as `hello.py`.

9. Python interpreter and interactive shell

An **interpreter** is the program that runs Python code. An **interactive shell**, also called a REPL, lets you enter one statement at a time and see results immediately.

REPL means **Read, Evaluate, Print, Loop**:

1. Read the statement.
2. Evaluate it.
3. Print a result when appropriate.
4. Wait for the next statement.

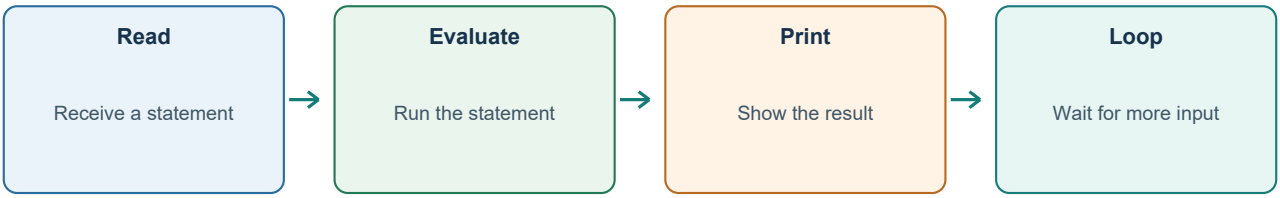
Example session:

SAMPLE OUTPUT

```
>>> 2 + 3
5
>>> name = "Mina"
>>> print(name)
Mina
```

- `>>>` is the primary prompt in the standard Python shell.
- `...` is a continuation prompt, often shown while entering an indented block.
- The shell is useful for short experiments.
- A `.py` file is better for saved, repeatable programs.

THE INTERACTIVE REPL CYCLE



Start the Windows Python shell with:

TERMINAL COMMAND

```
py
```

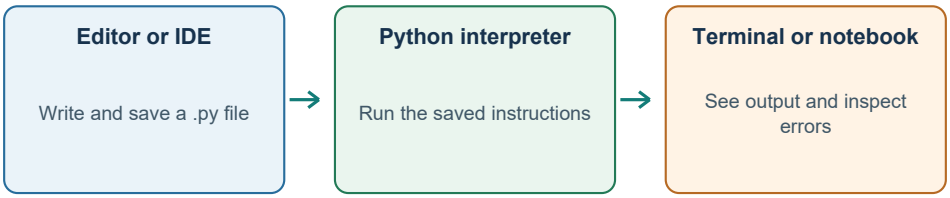
Leave it with `exit()` or the terminal's end-of-file shortcut. A program file and the interactive shell use the same Python language, but differ in how code is entered and saved.

10. IDEs: VS Code, PyCharm, and Jupyter Notebook

An editor is where you write text. An **IDE** adds development tools such as code completion, navigation, debugging, and project management. A **notebook** lets you organize executable code into cells alongside text and results.

Tool	Best for	Useful first check
VS Code	General-purpose editing, projects, and extensions	Select the intended Python interpreter in the workspace.
PyCharm	Python-focused projects and integrated navigation/debugging	Confirm the project interpreter and environment.
Jupyter Notebook	Experiments, teaching, visualizations, and data analysis	Select the intended kernel; a kernel is the running Python process for the notebook.

A SIMPLE PYTHON WORKFLOW



Choose the simplest tool that fits

- For the first few lessons, a terminal and a text editor are enough.
- Use an IDE when you want integrated help, debugging, or project navigation.
- Use a notebook when the work benefits from running cells and seeing output next to explanations.

An IDE is not Python itself. If a file runs in one terminal but not another, compare the selected Python interpreter and installed dependencies.

11. Your first Python program

Create a file named `hello.py` with:

PYTHON EXAMPLE

```
name = "Aisha"
print(f"Hello, {name}!")
```

Run it:

TERMINAL COMMAND

```
py hello.py
```

Output:

SAMPLE OUTPUT

```
Hello, Aisha!
```

What each line means

1. `name = "Aisha"` creates a name that refers to the string `"Aisha"`.
2. `f"Hello, {name}!"` creates a formatted string and inserts the value of `name`.
3. `print(...)` displays the result.

Algorithm

1. Store a person's name.
2. Build a greeting using that name.
3. Display the greeting.

Dry run

Step	`name`	Formatted text	Output
1	`"Aisha"`	-	-
2	`"Aisha"`	`"Hello, Aisha!"`	-
3	`"Aisha"`	`"Hello, Aisha!"`	`Hello, Aisha!`

Try changing `"Aisha"` to your own name and run the file again.

12. Python execution process

Running a file

When you run `py hello.py`, the operating system starts the Python launcher, the launcher selects an installed Python interpreter, and that interpreter executes the file.

A useful mental model

SAMPLE OUTPUT

```
Saved .py file
    -> Python reads and checks the code
    -> code is compiled to bytecode by CPython
    -> the Python virtual machine executes it
    -> output or an error is shown
```

The exact internal steps depend on the Python implementation and version.

Errors are part of execution

- A **syntax error** means Python cannot parse the code as valid syntax.
- A **runtime error** happens while a program is running, for example when it tries to divide by zero.
- A **traceback** shows where an error occurred and helps you find the relevant line.

Read the last line of a traceback first to identify the error type and message, then inspect the indicated line and the calls leading to it. Debugging is covered more fully in Chapter 21.

13. Source code, bytecode, and the Python virtual machine

Term	Definition	Analogy
Source code	The human-readable instructions in a `.py` file	The written recipe
Bytecode	A lower-level instruction format produced by a Python implementation	A recipe translated into a standard set of kitchen steps
Python virtual machine (PVM)	The runtime component that executes Python bytecode in CPython	The worker carrying out those translated steps

The PVM is not the same thing as a separate computer or a hardware processor. It is software in the Python runtime.

Different implementations can use different internal execution strategies. The terms "bytecode" and "PVM" are most useful here as a CPython learning model.

Remember the direction

SAMPLE OUTPUT

```
source code -> bytecode -> virtual-machine execution
```

You normally edit source code, not bytecode. If a `__pycache__` folder appears, Python created a cache for imported modules; do not rely on it as the program's source.

14. Comments and docstrings

Comments

A comment explains something to a human reader. A `#` starts a comment that continues to the end of the line.

PYTHON EXAMPLE

```
# Convert minutes to seconds.
minutes = 5
seconds = minutes * 60
print(seconds)
```

Comments do not make code run differently. Prefer comments that explain a reason or a non-obvious decision rather than restating the next line.

Docstrings

A docstring is a string placed at the start of a module, function, or class to document its purpose. Triple-quoted strings are commonly used.

PYTHON EXAMPLE

```
def minutes_to_seconds(minutes):
    """Convert a number of minutes to seconds."""
    return minutes * 60

print(minutes_to_seconds(5))
```

The function's docstring can be inspected:

PYTHON EXAMPLE

```
print(minutes_to_seconds.__doc__)
```

Difference: A comment is for readers of source code. A docstring is documentation attached to a module, class, or function and can be retrieved at runtime.

15. Python indentation and syntax

Indentation defines blocks

Python uses indentation to show which statements belong together. Four spaces per indentation level is the standard convention.

PYTHON EXAMPLE

```
temperature = 24

if temperature > 20:
    print("It is warm.")
    print("Consider drinking water.")

print("This line is outside the if block.")
```

The colon introduces the block. The indented lines belong to the `if`. The final `print` is not indented, so it runs after the conditional block.

Algorithm

1. Read the temperature.
2. Compare it with 20.
3. If it is greater than 20, display the two indented messages.
4. Display the final message in every case.

Dry run for `temperature = 24`

Step	Check or statement	Result
1	`24 > 20`	`True`
2	First indented `print`	`It is warm.`
3	Second indented `print`	`Consider drinking water.`
4	Unindented `print`	`This line is outside the if block.`

Common syntax mistakes

- Forgetting the colon after `if`, `for`, `while`, `def`, or `class`.
- Mixing tabs and spaces in a block.
- Using inconsistent indentation levels.
- Misspelling a name or using the wrong capitalization: `print` and `Print` are different names.
- Forgetting quotes around text: `print(hello)` looks for a variable named `hello`; `print("hello")` prints text.

Let your editor insert spaces consistently. If you see `IndentationError` or `SyntaxError`, read the line named in the error and also inspect the lines just before it.

16. `print()` and `input()`

`print()`

`print()` displays values. It can display more than one value, choose a separator with `sep`, and choose what to end with using `end`.

PYTHON EXAMPLE

```
print("Python", "is", "ready", sep=" - ")
print("One", end=" ")
print("line")
```

Output:

SAMPLE OUTPUT

```
Python - is - ready
One line
```

`input()`

`input()` displays an optional prompt, waits for the user to type text and press Enter, and returns that text as a string.

PYTHON EXAMPLE

```
name = input("What is your name? ")
print(f"Welcome, {name}!")
```

If a program needs a number, convert the returned string:

PYTHON EXAMPLE

```
age_text = input("How old are you? ")
age = int(age_text)
print(f"Next year you will be {age + 1}.")
```

`int()` works only when the text represents a valid integer. Invalid text causes a `ValueError`; robust error handling is taught in Chapter 15.

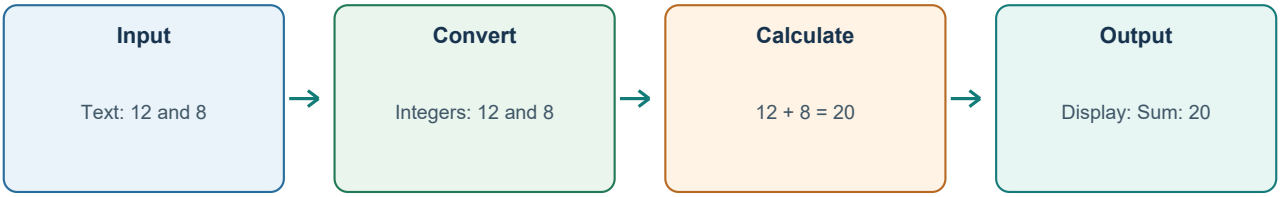
Worked algorithm: add two numbers

Problem: Ask for two whole numbers and display their sum.

Algorithm

1. Ask for the first number as text.
2. Convert it to an integer.
3. Ask for the second number as text.
4. Convert it to an integer.
5. Add the two integers.
6. Display the sum.

FROM INPUT TO ANSWER



Code

PYTHON EXAMPLE

```
first_text = input("First whole number: ")
first_number = int(first_text)

second_text = input("Second whole number: ")
second_number = int(second_text)

total = first_number + second_number
print(f"Sum: {total}")
```

Dry run with inputs `12` and `8`

Step	Variable or operation	Value
1	User types first input	`"12"` (text)
2	`int(first_text)`	`12` (integer)
3	User types second input	`"8"` (text)
4	`int(second_text)`	`8` (integer)
5	`12 + 8`	`20`
6	`print(...)`	`Sum: 20`

Key lesson: Input begins as text. Convert it before doing numeric arithmetic.

Common misconceptions

1. **"Python is only interpreted."** CPython normally compiles source to bytecode, then executes it in its runtime.
2. **"A variable has a permanent type."** In Python, a name can refer to values of different types at different times. The values still have types.
3. **"`input()` gives me a number."** It returns a string; convert it with `int()` or `float()` when needed.
4. **"An IDE is Python."** An IDE is a tool for editing and running code. A Python interpreter runs the program.
5. **"Indentation is just decoration."** In Python, indentation is part of the syntax and determines code blocks.
6. **"A comment and a docstring are identical."** A docstring is attached to a module, class, or function and can be inspected at runtime.

Practice problems with solutions

Problem 1: personal introduction

Ask for a name and a favorite subject. Print one sentence containing both.

Solution

PYTHON EXAMPLE

```
name = input("Name: ")
subject = input("Favorite subject: ")
print(f"{name} enjoys {subject}.")
```

Dry run: If the inputs are `Ravi` and `math`, the output is `Ravi enjoys math.`

Problem 2: minutes to seconds

Ask for a whole number of minutes and display the equivalent number of seconds.

Algorithm: Read minutes -> convert to integer -> multiply by 60 -> print.

PYTHON EXAMPLE

```
minutes = int(input("Minutes: "))
seconds = minutes * 60
print(f"Seconds: {seconds}")
```

Dry run: Input `3` gives `3 * 60 = 180`, so the output is `Seconds: 180`.

Problem 3: predict the output

PYTHON EXAMPLE

```
course = "Python"
print("Learn", course, sep=": ")
```

Solution: `print` receives two values and joins them using `": "`. The output is:

SAMPLE OUTPUT

```
Learn: Python
```

Problem 4: find and fix the syntax error

PYTHON EXAMPLE

```
if 5 > 2
    print("Five is greater")
```

Solution: The `if` statement needs a colon. Correct code:

PYTHON EXAMPLE

```
if 5 > 2:
    print("Five is greater")
```

Problem 5: explain the input type

What is the type of `value` in this code before conversion?

PYTHON EXAMPLE

```
value = input("Enter a number: ")
```

Solution: `value` is a string (`str`), even if the user types digits. For example, typing `42` gives the text `"42"`. Use `int(value)` to convert it to the integer `42`.

Key terms

Term	Short definition
Python	A high-level, general-purpose programming language
Implementation	A program that provides the language's behavior, such as CPython
Interpreter	Software that runs Python code
Interactive shell / REPL	A prompt for entering and evaluating statements interactively
Source code	Human-readable program text
Bytecode	A lower-level instruction representation used by an implementation
PVM	The Python virtual machine that executes bytecode in the CPython model
IDE	An editor combined with development tools
Notebook	A document containing executable cells, text, and outputs
Comment	A source-code note beginning with #
Docstring	A documentation string attached to a module, class, or function
Traceback	A report showing the call path and location associated with an error

Chapter mastery checklist

- ☐ I can explain Python in one or two sentences.
- ☐ I can name three strengths and one tradeoff of Python.
- ☐ I can describe the simplified CPython source-to-bytecode-to-VM path.

- ☐ I know the difference between a shell, a `.py` file, an IDE, and a notebook.
- ☐ I can save and run a Python file from a terminal.
- ☐ I can use comments, docstrings, and consistent four-space indentation.
- ☐ I know that `input()` returns text and can convert valid numeric input.
- ☐ I can trace the two-number addition program without running it.
- ☐ I completed the practice problems and can explain why each solution works.

Next chapter

Chapter 2 covers variables, assignment, naming rules, dynamic typing, object references, identity, and introductory memory concepts.

Chapter 02 - Variables, Constants and Memory

Level: Beginner **Suggested study time:** 4-6 hours **Prerequisites:** Chapter 1 - running a Python file and reading simple expressions

In this chapter, a variable is treated as a name that refers to an object, not as a permanent box with a type. That model makes assignment, dynamic typing, identity, and shared mutable values easier to understand.

Learning outcomes

- Create and update variables using assignment.
- Choose valid, readable names and recognize Python keywords.
- Explain dynamic typing and compare it with static typing.
- Use multiple assignment and unpacking safely.
- Explain the difference between equality and identity.
- Use `type()` and `id()` for basic inspection without making incorrect assumptions about memory.
- Describe introductory Python memory management and name scope.

1. What is a variable?

Definition

A variable is a name in a program that refers to a value or object. For example, after `score = 85`, the name `score` can be used to access the integer value `85`.

Easy explanation

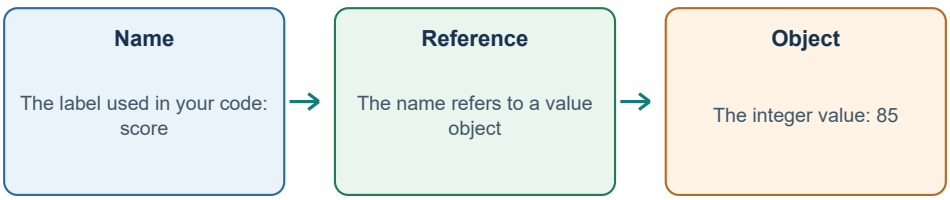
Imagine writing a label on a card and pointing the label at an object. The label is the variable name; the object is the value. The label does not permanently own a particular kind of value. It can later be made to refer to another object.

PYTHON EXAMPLE

```
score = 85
print(score)
```

`score` is the name. `85` is the integer object. The assignment statement binds the name to that object.

A NAME REFERS TO AN OBJECT



Important distinction

The = sign in an assignment means "bind the name on the left to the result on the right." It does not mean mathematical equality.

PYTHON EXAMPLE

```
score = 85
score = score + 5
print(score)
```

The second statement first calculates `score + 5`, which is `90`, and then rebinds `score` to the result.

Dry run

Step	Statement	Name binding after the step
1	`score = 85`	`score` refers to `85`
2	`score = score + 5`	Python reads `85`, calculates `90`, then binds `score` to `90`
3	`print(score)`	Displays `90`

2. Variable declaration and assignment

Definition

An assignment statement evaluates an expression and binds its result to one or more names. In ordinary Python code, you do not declare a variable's type before assigning to its name.

PYTHON EXAMPLE

```
student_name = "Nila"
student_score = 92
passed = True
```

Python creates the bindings when these statements run. The names are not available before their assignment in the current scope.

Assignment order

For `total = price * quantity`, Python:

1. Evaluates the expression `price * quantity`.
2. Gets the resulting value.
3. Binds `total` to that value.

If a name on the right has not been assigned, Python raises `NameError`.

PYTHON EXAMPLE

```
price = 4
quantity = 3
total = price * quantity
print(total)
```

Dry run

Step	Expression	Value
1	`price = 4`	`price` refers to `4`
2	`quantity = 3`	`quantity` refers to `3`
3	`price * quantity`	`4 * 3 = 12`

Step	Expression	Value
4	`total = ...`	`total` refers to `12`

Assignment versus comparison

- = assigns a value: `count = 3`
- == compares values: `count == 3`

They have different purposes and cannot be swapped.

3. Variable naming rules

Python names are case-sensitive identifiers. Use names that tell a reader what the value represents.

Rules

- A name can contain letters, digits, and underscores.
- A name cannot start with a digit.
- Spaces and punctuation such as `-` are not allowed in a variable name.
- Uppercase and lowercase are different: `score`, `Score`, and `SCORE` are three different names.
- A Python keyword cannot be used as a name.
- Python permits many Unicode letters, but English letters and underscores are the most portable choice for shared code.

Naming style

PYTHON EXAMPLE

```
student_name = "Nila"     # descriptive
quiz_score = 92           # descriptive
x = 92                    # valid, but unclear outside a small calculation
```

For ordinary variables and functions, Python's common style uses `snake_case`: lowercase words separated by underscores.

Name	Valid?	Reason
`user_name`	Yes	Letters and underscore
`score2`	Yes	A digit is allowed after the first character
`_total`	Yes	An underscore may begin a name
`2score`	No	A name cannot start with a digit
`user-name`	No	A hyphen is interpreted as subtraction
`first name`	No	Spaces are not allowed

Practical naming test

Before creating a name, ask: "Could someone who did not write this code guess what value it holds?" Prefer `monthly_income` to `m` when the purpose is not obvious.

4. Identifiers and keywords

An **identifier** is a name used for a variable, function, class, module, or other program object. A **keyword** is a reserved word with a special meaning in Python's grammar.

Examples of keywords include `if`, `else`, `for`, `while`, `def`, `class`, `return`, `True`, `False`, and `None`.

Do not memorize a list from a random webpage; the language can add keywords. Ask Python for the list when needed:

PYTHON EXAMPLE

```
import keyword

print(keyword.kwlist)
```

You can also check one name:

PYTHON EXAMPLE

```
import keyword

print(keyword.iskeyword("class"))
print(keyword.iskeyword("student_name"))
```

The first call displays `True`; the second displays `False`. A built-in name such as `list` is not necessarily a keyword, but replacing it with your own variable name is still a bad idea because it hides the built-in function.

5. Dynamic typing

Definition

Python is dynamically typed: values have types at runtime, and a name can be rebound to values of different types during execution.

PYTHON EXAMPLE

```
item = 25
item = "twenty-five"
```

This is valid. In the first assignment, `item` refers to an integer. In the second, it refers to a string.

Dynamic typing does **not** mean that Python values have no types. It means you do not normally write a type declaration for every variable before using it.

Dry run

Statement	Value referred to by `item`	Type
`item = 25`	`25`	`int`
`item = "twenty-five"`	`"twenty-five"`	`str`

If a later operation is not supported by the current value, Python reports an error:

PYTHON EXAMPLE

```
item = "twenty-five"
print(item + 5)
```

This raises `TypeError` because Python does not know whether the programmer intended text concatenation, numeric conversion, or something else.

6. Static vs dynamic typing

Definition

- In a **statically typed** language, type checking is commonly performed before the program runs, and variables or expressions have declared or inferred types that constrain valid operations.
- In a **dynamically typed** language, types are associated with runtime values and many type checks happen while the program runs.

The exact rules vary among languages; these terms describe broad approaches, not a complete comparison.

Question	Dynamic typing (Python)	Static typing (for example, Java)
When are many type mistakes detected?	Often while the relevant code runs	Often during compilation or static analysis
Must a beginner declare every variable type?	Usually no	Often yes, depending on the language
Can names refer to different types of values?	Yes	Usually restricted by the declared or inferred type
Does either approach prevent every bug?	No	No

Python also supports optional type annotations and external static type checkers. These tools are covered in Chapter 23. Python's runtime behavior remains dynamically typed.

7. Multiple variable assignments

Python lets one assignment statement bind several names.

Chained assignment

PYTHON EXAMPLE

```
first = second = 0
```

Both names refer to the same integer object. This is safe for integers because integers are immutable. Do not assume the same pattern creates independent mutable lists:

PYTHON EXAMPLE

```
first = second = []
first.append("new")
print(second)
```

The output is `['new']` because both names refer to the same list. Use separate list literals when independent lists are intended:

PYTHON EXAMPLE

```
first = []
second = []
```

Assigning different values

PYTHON EXAMPLE

```
minimum, maximum = 0, 100
```

The right-hand expressions are evaluated, then assigned to the left-hand names from left to right.

8. Multiple-value unpacking

Unpacking assigns items from an iterable to several names.

PYTHON EXAMPLE

```
first, second = 10, 20
print(first)
print(second)
```

The number of targets must match the number of values unless a starred target collects the remainder.

Swap two names

Python can swap names without a temporary variable:

PYTHON EXAMPLE

```
left = "A"
right = "B"
left, right = right, left
print(left, right)
```

Algorithm

1. Evaluate the right-hand values using the current bindings: `right` is `"B"` and `left` is `"A"`.
2. Keep those two results for the assignment.
3. Bind `left` to `"B"`.
4. Bind `right` to `"A"`.

Dry run

Moment	`left`	`right`
Before swap	`"A"`	`"B"`
Values evaluated on right	`"A"`	`"B"`
After unpacking assignment	`"B"`	`"A"`

Starred unpacking

PYTHON EXAMPLE

```
first, *middle, last = [10, 20, 30, 40]
print(first)
print(middle)
print(last)
```

The result is `first == 10`, `middle == [20, 30]`, and `last == 40`. The starred target receives a list. A starred target can collect zero or more values.

If the number of values does not fit the targets, Python raises `ValueError`:

PYTHON EXAMPLE

```
first, second = [10, 20, 30]  # too many values
```

9. Constants and naming conventions

Python does not have a general syntax that makes a variable permanently constant. By convention, a name written in uppercase with underscores signals that a value is intended to remain unchanged.

PYTHON EXAMPLE

```
SECONDS_PER_MINUTE = 60
MAX_RETRIES = 3
```

This convention helps readers. It does not stop reassignment:

PYTHON EXAMPLE

```
SECONDS_PER_MINUTE = 60
SECONDS_PER_MINUTE = 30  # Python allows this; avoid it by convention.
```

For type-checking tools, `typing.Final` can communicate that a name should not be reassigned:

PYTHON EXAMPLE

```
from typing import Final

SECONDS_PER_MINUTE: Final = 60
```

`Final` is a static-analysis hint; it does not make the value immutable at runtime.

Three different ideas

- **Constant name:** A naming convention that tells people not to reassign a name.
- **Immutable object:** An object whose value cannot be changed in place, such as an `int` or `str`.
- **Mutable object:** An object that can be changed in place, such as a `list` or `dict`.

A constant name may refer to a mutable object. The name convention alone does not freeze that object.

10. Object references

Definition

An object reference is the relationship between a name and the object it currently refers to. Assignment usually creates or updates a binding; it does not automatically make a copy of the object.

PYTHON EXAMPLE

```
original = [1, 2]
alias = original
alias.append(3)
print(original)
```

Both names refer to the same list, so changing the list through `alias` is visible through `original`.

Dry run

Step	Operation	`original` and `alias`
1	Create `[1, 2]` and bind `original`	`original -> list [1, 2]`
2	`alias = original`	Both names refer to that list
3	`alias.append(3)`	The shared list becomes `[1, 2, 3]`
4	`print(original)`	Displays `[1, 2, 3]`

To create a shallow copy of a list, use a copy operation:

PYTHON EXAMPLE

```
original = [1, 2]
copy_of_original = original.copy()
copy_of_original.append(3)
print(original)  # [1, 2]
```

Shallow and deep copying are studied further with collections in later chapters.

11. Object identity and `id()`

Equality and identity preview

Every object has an identity during its lifetime. `id(value)` returns an integer that identifies that object for the lifetime of the object. The exact representation is implementation-dependent.

PYTHON EXAMPLE

```
value = [1, 2]
print(id(value))
```

Do not treat the number from `id()` as a portable memory address. Do not assume a particular number, pattern, or ordering. An identity may be reused after an object is no longer alive.

Use `id()` mainly as a learning or debugging tool when investigating whether two references point to the same object. In ordinary programs, `is` is the direct identity test:

PYTHON EXAMPLE

```
first = []
second = first
third = []

print(first is second)  # True
print(first is third)   # False
```

`first` and `second` share one object; `third` is a separate list with the same initial contents.

12. The `type()` function

`type(value)` returns the type object of a value. It is useful for learning and basic inspection.

PYTHON EXAMPLE

```
print(type(42))
print(type(3.5))
print(type("hello"))
print(type([1, 2]))
```

Typical output names are `int`, `float`, `str`, and `list`.

For a yes/no question such as "is this an integer or a subclass of integer?", `isinstance(value, int)` is usually more flexible than comparing `type(value)` exactly. The distinction matters because Python has type inheritance; for example, `bool` is a subclass of `int`.

PYTHON EXAMPLE

```
print(isinstance(True, int))  # True
```

This behavior is explained more fully in the data-types chapter.

13. `is` vs `==`

These operators answer different questions:

- `a == b` asks whether the values compare equal.
- `a is b` asks whether both names refer to the same object.

PYTHON EXAMPLE

```
first = [1, 2]
second = [1, 2]
third = first

print(first == second)  # True: same contents
print(first is second)  # False: separate list objects
print(first is third)   # True: same object
```

Correct use of `is`

Use `is` when identity is what matters, especially when testing for `None`:

PYTHON EXAMPLE

```
result = None

if result is None:
    print("No result yet")
```

Do not use `is` to compare strings or numbers for their value. Some implementations reuse certain objects as an optimization, so identity results for equal small values should not be used as a substitute for `==`.

Dry run

Expression	Question	Result
`first == second`	Do their list values compare equal?	`True`
`first is second`	Are they the same list object?	`False`
`first is third`	Are they the same list object?	`True`

14. Memory management basics

Python's runtime manages memory for objects. As a beginner, create values and let Python manage their ordinary lifetime; do not try to manually free memory.

Useful ideas

- Names are bindings in a namespace.
- Objects occupy memory while they are needed by the running program.
- Multiple names can refer to one object.
- Removing one name does not necessarily remove the object if another name still refers to it.
- When an object is no longer reachable and is no longer needed, the implementation can reclaim its memory.

In CPython, reference counting is an important memory-management mechanism, and a cyclic garbage collector handles certain reference cycles. These are implementation details, not a guarantee that every Python implementation uses exactly the same internal mechanism.

`del` removes a binding

PYTHON EXAMPLE

```
numbers = [1, 2]
another_name = numbers
del numbers
print(another_name)
```

`del numbers` removes the name `numbers`. The list remains reachable through `another_name`, so it is still usable.

Practical memory habits

- Avoid keeping large objects reachable after they are no longer useful.
- Do not create unnecessary copies of very large data.
- Use a context manager for resources such as files; file handling is covered in Chapter 16.
- Profile memory when a real program has a measured memory problem instead of guessing.

15. Variable scope introduction

Definition

Scope describes where a name can be accessed. A **local variable** is created inside a function and is normally available only within that function. A name defined at the top level of a module is a **global name** for that module.

PYTHON EXAMPLE

```
course_name = "Python"  # module-level name

def show_course():
    message = "Learning " + course_name  # local name: message
    print(message)

show_course()
```

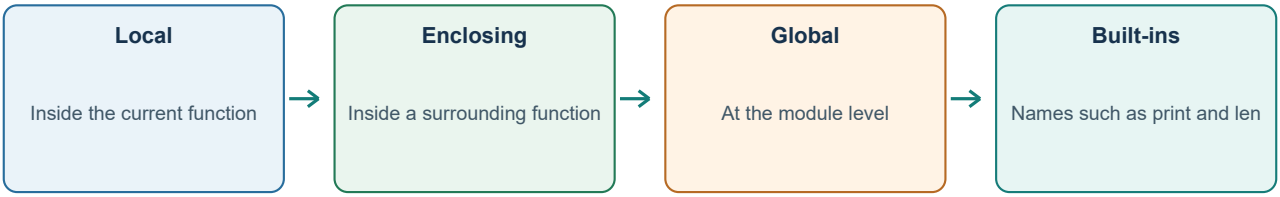
`message` is local to `show_course`. `course_name` is found at module scope.

LEGB lookup rule

When Python looks up a name inside a function, the common search order is:

1. **L - Local:** The current function.
2. **E - Enclosing:** Any enclosing function, from inner to outer.
3. **G - Global:** The current module.
4. **B - Built-in:** Names provided by Python, such as `print` and `len`.

WHERE PYTHON LOOKS FOR A NAME



Local variables and parameters

PYTHON EXAMPLE

```
def add_tax(price, tax_rate):
    tax = price * tax_rate
    return price + tax

total = add_tax(100, 0.1)
print(total)
```

`price`, `tax_rate`, and `tax` are local to the function call. The `return` statement sends a result back, where it is bound to the outer name `total`.

Scope mistakes

Do not use a global variable when a function parameter or return value communicates the data more clearly. The `global` and `nonlocal` keywords exist, but using them casually can make programs harder to reason about. Functions and scope are covered in depth in Chapter 12.

Worked program: swap two values and report the types

Problem

Start with two values, swap their references using unpacking, then display the values and their types.

Algorithm

1. Bind `left` to `"red"` and `right` to `"blue"`.
2. Evaluate the right-hand side `right, left` using the current values.
3. Assign those two values to `left` and `right`.
4. Display the new values.
5. Display the type of each value.

PYTHON EXAMPLE

```
left = "red"
right = "blue"

left, right = right, left

print(left, right)
print(type(left).__name__, type(right).__name__)
```

Dry run

Step	`left`	`right`	Notes
1	`"red"`	-	First assignment
2	`"red"`	`"blue"`	Second assignment
3	`"blue"`	`"red"`	Unpacking swap
4	-	-	Prints `blue red`
5	-	-	Prints `str str`

Why the swap works

Python evaluates the values on the right before binding the targets on the left. The original values are therefore available while the new bindings are made.

Common mistakes and debugging

`NameError`

PYTHON EXAMPLE

```
print(total)
```

If `total` has not been assigned in the current scope, Python raises `NameError`. Check spelling, capitalization, and whether the assignment ran before the use.

Accidental reuse of a built-in name

PYTHON EXAMPLE

```
list = [1, 2, 3]
```

This hides the built-in `list` name in the current scope. Choose a descriptive name such as `scores`.

Unexpected shared list

PYTHON EXAMPLE

```
first = second = []
first.append(1)
```

Both names refer to one list. Create two separate lists if that is what the problem requires.

Comparing identity instead of value

PYTHON EXAMPLE

```
name = input("Name: ")
if name is "Aisha":  # do not use identity for string value comparison
    print("Hello")
```

Use `==` to compare string contents:

PYTHON EXAMPLE

```
if name == "Aisha":
    print("Hello")
```

Practice problems with solutions

Problem 1: calculate a rectangle's area

Use `width = 8` and `height = 5`. Store the area in a well-named variable and print it.

Solution

PYTHON EXAMPLE

```
width = 8
height = 5
rectangle_area = width * height
print(rectangle_area)
```

The multiplication gives `40`. `rectangle_area` describes what the result represents.

Problem 2: swap two numbers

Start with `first = 4` and `second = 9`. Swap them and print both.

Solution

PYTHON EXAMPLE

```
first = 4
second = 9
first, second = second, first
print(first, second)
```

Output: `9 4`.

Problem 3: identify aliasing

What does this program print, and why?

PYTHON EXAMPLE

```
one = [5]
two = one
two.append(8)
print(one)
```

Solution: It prints `[5, 8]`. Both names refer to the same list object, and `append` changes that object.

Problem 4: choose equality or identity

Two separately created lists contain the same values. Which operator checks that their contents compare equal?

Solution: Use `==`. Use `is` only when you need to know whether both names refer to the exact same object.

Problem 5: fix a naming error

Why is `2nd_score = 90` invalid? Give a valid alternative.

Solution: A Python identifier cannot begin with a digit. Use `second_score = 90`.

Key terms

Term	Short definition
Variable / name	A name bound to a value or object
Assignment	Evaluation of a right-hand expression followed by a name binding
Identifier	A name used for a program object
Keyword	A reserved word in the Python language
Dynamic typing	Runtime values have types; names can be rebound to other types
Constant convention	Uppercase naming that signals a value should not be reassigned
Reference	A name-to-object relationship
Identity	The property that distinguishes one object from another during its lifetime
`id()`	A function returning an object's identity value
Scope	The region where a name can be accessed
LEGB	Local, Enclosing, Global, Built-in name lookup order

Chapter mastery checklist

- ☐ I can explain a variable as a name bound to an object.
- ☐ I can use = for assignment and == for value comparison.
- ☐ I can identify valid names and avoid Python keywords.
- ☐ I can explain dynamic typing without saying values have no types.
- ☐ I can unpack values and swap two names.
- ☐ I understand why `first = second = []` creates an alias.
- ☐ I know that `is` checks identity and == compares values.
- ☐ I can use `type()` and `isinstance()` to inspect values.
- ☐ I can explain local scope and the LEGB lookup order.
- ☐ I can dry-run the worked swap program.

Next chapter

Chapter 3 introduces Python's built-in data types, including numbers, strings, lists, tuples, ranges, sets, dictionaries, and `None`.

Chapter 03 - Python Data Types

Level: Beginner **Suggested study time:** 5-7 hours **Prerequisites:** Chapters 1 and 2

A data type describes the kind of value an object represents and the operations available for it. Choosing the right type helps a program store, validate, and process information correctly.

Learning outcomes

- Recognize Python's common built-in types.
- Use numeric, Boolean, and string values.
- Choose between lists, tuples, ranges, sets, and dictionaries.
- Explain mutability, ordering, and hashability.
- Check types and convert values safely.
- Diagnose common type errors by examining the values and operations involved.

1. What are data types?

Definition

A data type describes the kind of value an object represents and the operations that make sense for that value.

Examples:

- `42` is an integer; it can be added to another integer.
- `"42"` is text; it can be joined to other text.
- `[4, 2]` is a list; it can store multiple values and can be changed.
- `{"name": "Mina"}` is a dictionary; it maps keys to values.

Easy explanation

Think of a type as a set of rules for how a value behaves. The characters `42` can mean a number or text depending on how they are written:

PYTHON EXAMPLE

```
number = 42
text = "42"

print(number + 1)  # 43
print(text + "1")  # 421
```

The visible digits look similar, but their types support different operations.

Dry run

Expression	Type of values	Operation	Result
`42 + 1`	`int` and `int`	Numeric addition	`43`
`"42" + "1"`	`str` and `str`	Text concatenation	`"421"`

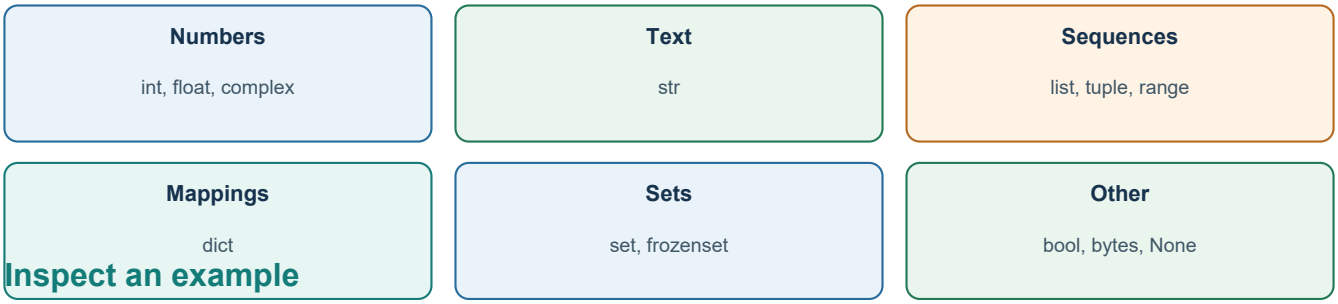
2. Built-in data types

Python provides common types as part of the language. This table groups important examples by purpose.

Group	Common built-in types	Typical use
Numbers	`int`, `float`, `complex`	Counts, measurements, and mathematical values
Truth values	`bool`	Conditions that are true or false
Text	`str`	Names, messages, and other Unicode text
Sequences	`list`, `tuple`, `range`	Ordered values that can be visited by position
Mapping	`dict`	Key-to-value associations
Sets	`set`, `frozenset`	Collections of unique hashable values
Binary data	`bytes`, `bytearray`, `memoryview`	Data represented as bytes
Absence	`NoneType`	The single value `None`, often meaning "no value"

Python also has types in libraries, such as `Decimal`, `date`, and `DataFrame`. Those are not all built-in types.

PYTHON'S EVERYDAY BUILT-IN TYPES



Inspect an example

PYTHON EXAMPLE

```
values = [42, 3.5, True, "hello", [1, 2], (1, 2), {1, 2}, {"x": 1}, None]

for value in values:
    print(value, type(value).__name__)
```

This program loops over several values and prints each value with its type name. Loops are taught in detail in Chapter 6; here, focus on the different types in the list.

3. Numeric types: `int`, `float`, and `complex`

Integers: `int`

An integer represents a whole number without a fractional part.

PYTHON EXAMPLE

```
students = 28
temperature_change = -3
```

Python integers can grow beyond the fixed-width range used by many processors, limited in practice by available memory. Very large integer calculations may take more time and memory.

Floating-point numbers: `float`

A float represents an approximate real number using binary floating-point arithmetic.

PYTHON EXAMPLE

```
price = 19.99
ratio = 3.0
```

Some decimal fractions cannot be represented exactly in binary floating point:

PYTHON EXAMPLE

```
print(0.1 + 0.2)
```

The result may display as `0.30000000000000004`. For many measurements this approximation is acceptable. For exact decimal currency rules, Python's `decimal.Decimal` may be a better tool.

Complex numbers: `complex`

A complex number has real and imaginary parts. Python uses `j` for the imaginary unit:

PYTHON EXAMPLE

```
signal = 2 + 3j
print(signal.real)
print(signal.imag)
```

The output is `2.0` and `3.0`. Complex numbers are useful in some engineering, scientific, and mathematical applications.

Arithmetic example

PYTHON EXAMPLE

```
whole = 7
measurement = 2.5
combined = whole + measurement
print(combined)
print(type(combined).__name__)
```

The result is `9.5`, and its type is `float`. When an integer and a float are combined with addition, Python converts the integer for that operation.

4. Boolean: `True` and `False`

A Boolean (`bool`) represents one of two truth values: `True` or `False`. Booleans are often produced by comparisons and used by conditions.

PYTHON EXAMPLE

```
age = 20
is_adult = age >= 18
print(is_adult)
```

The comparison `20 >= 18` is true, so `is_adult` refers to `True`.

Truthiness

Some values behave as false in a condition, such as `False`, `None`, numeric zero, and empty collections. Many other values behave as true. The conversion can be inspected with `bool()`:

PYTHON EXAMPLE

```
print(bool(0))       # False
print(bool(5))       # True
print(bool(""))      # False
print(bool("False")) # True: this is non-empty text
```

Important: `bool("False")` is `True` because the string is not empty. It does not interpret the word as a Boolean setting. Text-to-Boolean parsing needs an explicit rule, covered in the conversion section.

In Python, `bool` is a subclass of `int`, so `True` behaves like `1` and `False` like `0` in some numeric operations. In beginner programs, use Boolean values for logic rather than as a replacement for numeric values.

5. String: `str`

A string is an immutable sequence of Unicode text.

PYTHON EXAMPLE

```
name = "Mina"
message = 'Welcome!'
multiline = """This text
uses more than one line."""
```

Single and double quotes both create strings. Triple quotes are commonly used for multiline strings and docstrings.

Useful operations

PYTHON EXAMPLE

```
first = "data"
second = "science"

print(first + " " + second)
print(first.upper())
print(len(second))
print(first[0])
```

Results:

- Concatenation creates `"data science"`.
- `.upper()` returns `"DATA"`.
- `len(second)` returns `7`.
- `first[0]` returns `"d"`.

Indexing starts at zero. Since strings are immutable, an item cannot be changed in place:

PYTHON EXAMPLE

```
word = "cat"
# word[0] = "b"  # TypeError: strings do not support item assignment
word = "b" + word[1:]
print(word)  # bat
```

The last line creates a new string and rebinds `word`.

Unicode and encoding

Python strings represent Unicode text. Encoding is the process of converting text to bytes for storage or communication. UTF-8 is a common encoding:

PYTHON EXAMPLE

```
message = "cafe"
encoded = message.encode("utf-8")
decoded = encoded.decode("utf-8")
print(decoded)
```

Text and bytes are different types. File and network programs must use an agreed encoding when converting between them.

6. Sequence types

A sequence is an ordered collection whose items can be accessed by position. Common built-in sequences include `str`, `list`, `tuple`, and `range`.

Sequences commonly support:

- Indexing: `items[0]`
- Slicing: `items[1:3]`
- Length: `len(items)`
- Membership: `value in items`
- Iteration: visiting items one at a time

PYTHON EXAMPLE

```
letters = ["a", "b", "c", "d"]
print(letters[0])       # a
print(letters[1:3])     # ['b', 'c']
print("c" in letters)   # True
```

Slicing includes the starting index and excludes the ending index. `letters[1:3]` therefore contains indexes 1 and 2.

Not every sequence is mutable. Lists can be changed in place; strings and tuples cannot.

7. List

A list is an ordered, mutable collection. It can hold values of different types, although lists with one clear purpose are often easier to use.

PYTHON EXAMPLE

```
scores = [72, 81, 90]
scores.append(95)
scores[0] = 75
print(scores)
```

The final list is `[75, 81, 90, 95]`.

Common list operations

Operation	Example	Effect
Create	`items = [1, 2]`	Creates a list
Add to end	`items.append(3)`	Adds one item
Read by index	`items[0]`	Returns the first item
Replace by index	`items[0] = 9`	Changes an item
Remove by value	`items.remove(2)`	Removes the first matching value
Get length	`len(items)`	Counts the items

Mutability and references

PYTHON EXAMPLE

```
first = [1, 2]
second = first
second.append(3)
print(first)
```

This prints `[1, 2, 3]` because `first` and `second` refer to the same mutable list.

Create a shallow copy when a separate outer list is needed:

PYTHON EXAMPLE

```
first = [1, 2]
second = first.copy()
second.append(3)
print(first)   # [1, 2]
print(second)  # [1, 2, 3]
```

The copy is shallow: nested mutable objects inside the list may still be shared.

8. Tuple

A tuple is an ordered, immutable collection. Use it when the group of values should not be changed as a collection.

PYTHON EXAMPLE

```
point = (4, 7)
print(point[0])
```

The result is `4`. This would fail because a tuple does not support item assignment:

PYTHON EXAMPLE

```
point = (4, 7)
# point[0] = 9  # TypeError
```

One-item tuple

A comma creates a one-item tuple:

PYTHON EXAMPLE

```
one_item = ("Python",)
not_a_tuple = ("Python")
print(type(one_item).__name__)     # tuple
print(type(not_a_tuple).__name__) # str
```

The parentheses alone do not make a tuple; the comma does.

Immutability is shallow

A tuple cannot replace its own element, but an element inside it may refer to a mutable object:

PYTHON EXAMPLE

```
record = ("scores", [70, 80])
record[1].append(90)
print(record)
```

This is allowed because the list changes; the tuple's reference to that list remains the same. Immutable tuple does not mean every nested value is immutable.

9. `range`

`range` represents an arithmetic sequence of integers. It is often used to repeat an operation a known number of times.

PYTHON EXAMPLE

```
for number in range(1, 5):
    print(number)
```

This displays `1`, `2`, `3`, and `4`. The stop value `5` is excluded.

Forms

- `range(stop)` starts at 0 and steps by 1.
- `range(start, stop)` starts at `start` and stops before `stop`.
- `range(start, stop, step)` also sets the increment.

PYTHON EXAMPLE

```
print(list(range(2, 11, 2)))
```

The result is `[2, 4, 6, 8, 10]`. Converting to `list` is useful for seeing the values; a range itself stores the parameters and generates values as needed.

Dry run for `range(2, 8, 2)`

Attempted value	Is it less than stop `8`?	Included?
2	Yes	Yes
4	Yes	Yes
6	Yes	Yes
8	No	No

10. Set and `frozenset`

A set is a collection of unique, hashable values. A set is mutable; a `frozenset` is its immutable counterpart.

PYTHON EXAMPLE

```
tags = {"python", "data", "python"}
print(tags)
```

The set contains only `"python"` and `"data"`; repeated values are collapsed. Do not rely on set display or iteration order.

Common set operations

PYTHON EXAMPLE

```
left = {"a", "b", "c"}
right = {"b", "c", "d"}

print(left | right)  # union
print(left & right)  # intersection
print(left - right)  # values only in left
```

The output order may vary because sets are not an indexed sequence.

Empty set

PYTHON EXAMPLE

```
empty_set = set()
empty_dictionary = {}
```

`{}` creates a dictionary, not a set. Use `set()` for an empty set.

Immutable set

PYTHON EXAMPLE

```
fixed_tags = frozenset({"python", "data"})
```

A `frozenset` cannot be changed after creation and can be used as a dictionary key or as a member of another set if its contents are hashable.

11. Dictionary

A dictionary (`dict`) maps keys to values. Keys are unique and must be hashable. Values can be of any type.

PYTHON EXAMPLE

```
student = {
    "name": "Mina",
    "score": 92,
}

print(student["name"])
student["score"] = 95
student["passed"] = True
```

`student["name"]` looks up the value for the `"name"` key. Assignment updates an existing key or adds a new one.

Safer lookup with `get`

Indexing a missing key raises `KeyError`. `get` returns a default when a key is absent:

PYTHON EXAMPLE

```
student = {"name": "Mina"}
score = student.get("score", 0)
print(score)
```

The output is `0`. `get` does not add the missing key.

Iterating over dictionary data

PYTHON EXAMPLE

```
for key, value in student.items():
    print(key, value)
```

Dictionary insertion order is guaranteed by the language specification in modern Python versions (Python 3.7 and later). This does not make a dictionary a sequence: it is accessed by keys, not integer positions.

12. `None`

`None` is Python's single value meaning "no value" or "not available here." Its type is `NoneType`.

PYTHON EXAMPLE

```
result = None

if result is None:
    print("No result has been calculated.")
```

Use `is None` and `is not None` for this check. Do not use `== None` as the normal style.

Functions that finish without an explicit `return` return `None`. A function may also return `None` intentionally to indicate that no result was found.

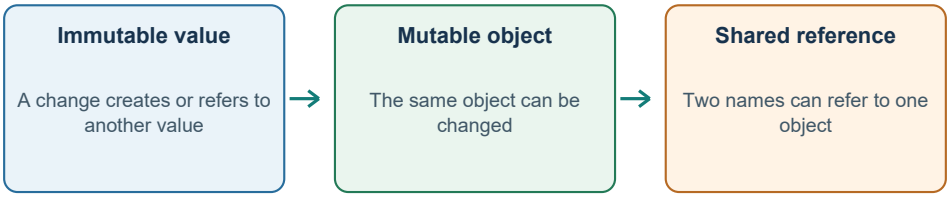
`None` is not the same as `0`, `False`, or an empty string. Those values have different meanings and types.

13. Mutable vs immutable

- A **mutable** object can be changed after it is created. Examples include lists, dictionaries, and sets.

- An **immutable** object cannot be changed in place. Examples include integers, floats, Booleans, strings, and tuples as containers.

MUTABLE AND IMMUTABLE VALUES



Rebinding is not mutation

PYTHON EXAMPLE

```
count = 4
count = count + 1
```

The integer `4` was not changed. The expression produces an integer value `5`, and the name `count` is rebound.

PYTHON EXAMPLE

```
names = ["Mina"]
names.append("Ravi")
```

Here, the existing list object is mutated by adding an item.

Why the difference matters

When two names refer to one mutable object, a mutation through either name is visible through both. With an immutable value, an operation produces a result and a name can be rebound to it.

14. Ordered vs unordered

An ordered collection has a defined order that can be observed consistently according to its type's rules. A sequence such as a list preserves its item positions.

- `list`, `tuple`, `str`, and `range` have sequence order.
- Dictionaries preserve insertion order in modern Python.
- Sets do not promise a useful iteration order; do not write code that depends on one.

Important: "Unordered" does not mean the display must be random every time. It means the type does not promise the positional ordering that your program should rely on.

If a stable sorted result is needed, sort the values explicitly:

PYTHON EXAMPLE

```
numbers = {8, 2, 5}
print(sorted(numbers))
```

`sorted` returns a list in ascending order.

15. Hashable vs unhashable

An object is **hashable** when it has a hash value that remains stable during its lifetime and it can be compared for equality consistently. Hashable values can be used as dictionary keys and set members.

Common hashable values include integers, strings, and tuples whose elements are all hashable. Lists, dictionaries, and sets are unhashable because they are mutable.

PYTHON EXAMPLE

```
student_scores = {
    "Mina": 92,
    "Ravi": 88,
}
```

Strings work as keys. This does not:

PYTHON EXAMPLE

```
# invalid = {[1, 2]: "value"}  # TypeError: list is unhashable
```

A tuple is hashable only when all of its contents are hashable:

PYTHON EXAMPLE

```
valid_key = (1, "room")
invalid_key = (1, [2, 3])
```

The first tuple can be used as a key; the second cannot because it contains a list.

16. Type checking

Use `type(value)` to inspect the exact type during learning:

PYTHON EXAMPLE

```
value = 12.5
print(type(value))
```

Use `isinstance(value, expected_type)` for a general type check that also recognizes subclasses:

PYTHON EXAMPLE

```
value = True
print(isinstance(value, bool))  # True
print(isinstance(value, int))   # True: bool is a subclass of int
```

Do not scatter checks through a program when the operations themselves can clearly express the requirement. Validate input at boundaries, then keep the main logic straightforward.

Exact type vs compatible type

- `type(value) is int` asks whether the exact type is `int`.
- `isinstance(value, int)` asks whether it is an `int` or an instance of a subclass.

For most ordinary type validation, prefer `isinstance`.

17. Type conversion

Type conversion creates or requests a value of another type. Common constructors include `int`, `float`, `str`, `bool`, `list`, `tuple`, and `set`.

PYTHON EXAMPLE

```
age_text = "21"
age = int(age_text)

price_text = "12.5"
price = float(price_text)

print(age + 1)
print(price * 2)
```

Not every conversion is possible:

PYTHON EXAMPLE

```
# int("twenty")  # ValueError
```

Converting collections

PYTHON EXAMPLE

```
letters = list("cat")
unique_letters = set("cacao")
pair = tuple([1, 2])
```

Results:

- `list("cat")` creates `["c", "a", "t"]`.
- `set("cacao")` contains unique letters, with no promised order.
- `tuple([1, 2])` creates `(1, 2)`.

Boolean conversion warning

`bool(value)` checks truthiness; it does not parse words such as `"true"` or `"false"`:

PYTHON EXAMPLE

```
print(bool("false"))  # True, because the string is non-empty
```

For text settings, define accepted spellings and compare normalized text:

PYTHON EXAMPLE

```
answer = input("Enable feature? ").strip().lower()
enabled = answer in {"yes", "y", "true"}
print(enabled)
```

The braces above create a set of accepted strings; membership is used to check whether the normalized answer matches one of them.

18. Implicit vs explicit conversion

Implicit conversion

An implicit conversion is performed by Python as part of an operation when the language defines a safe rule for it.

PYTHON EXAMPLE

```
result = 3 + 2.5
print(result)
print(type(result).__name__)
```

Python converts the integer to a floating-point value for the addition. The result is `5.5` and has type `float`.

Explicit conversion

An explicit conversion is requested by the programmer:

PYTHON EXAMPLE

```
age_text = "21"
age = int(age_text)
```

Explicit conversion makes an important decision visible. It can also fail, so untrusted input may need validation and exception handling.

Do not assume Python will convert arbitrary text to a number or automatically treat a numeric-looking string as numeric.

19. Type errors and debugging

A `TypeError` often means an operation is not defined for the types or values used.

PYTHON EXAMPLE

```
age_text = "21"
next_age = age_text + 1
```

The program tries to combine a string and an integer. Python raises `TypeError`.

Debugging method

1. Read the exception type and message at the end of the traceback.
2. Locate the line where the operation failed.
3. Inspect the values immediately before the operation.
4. Use `type(value)` or `isinstance(value, Type)` if you are unsure.
5. Decide the intended meaning: text concatenation, numeric arithmetic, or another operation.
6. Convert or validate at the point where external data enters the program.

Corrected example:

PYTHON EXAMPLE

```
age_text = "21"
age = int(age_text)
next_age = age + 1
print(next_age)
```

Output: `22`.

Practical checklist

- Is the input a string because it came from `input()`?
- Is a list being changed through another name that refers to the same object?
- Is a dictionary key missing?
- Is a set being treated like an indexed list?
- Is a conversion invalid for the actual input text?
- Does the traceback identify a different line than the one you first suspected?

Worked algorithm: remove duplicates but preserve first-seen order

Problem

Given a list of words, create a new list with each word appearing once, preserving the order of its first appearance.

Algorithm

1. Start with an empty set called `seen` and an empty list called `unique_words`.
2. Visit each word in the input list from left to right.
3. If the word is not in `seen`, add it to `seen` and append it to `unique_words`.
4. Return or display `unique_words`.

The list preserves order. The set makes membership checks convenient. A set's own order is not used for the result.

PYTHON EXAMPLE

```
words = ["red", "blue", "red", "green", "blue"]
seen = set()
unique_words = []

for word in words:
    if word not in seen:
        seen.add(word)
        unique_words.append(word)

print(unique_words)
```

Dry run

Input word	Already in `seen`?	`seen` after step	`unique_words` after step
`"red"`	No	`{"red"}`	`["red"]`
`"blue"`	No	`{"red", "blue"}`	`["red", "blue"]`
`"red"`	Yes	unchanged	unchanged
`"green"`	No	includes `"green"`	`["red", "blue", "green"]`
`"blue"`	Yes	unchanged	unchanged

Output:

SAMPLE OUTPUT

```
['red', 'blue', 'green']
```

This example uses both a mutable list and a mutable set, and demonstrates why the set's ordering is irrelevant to the required result.

Common misconceptions

1. **"A number written with digits is always numeric."** Quotes make `"42"` a string.
2. **"A tuple makes everything inside it immutable."** A tuple's references cannot be replaced, but a referenced list can still be changed.
3. **"Sets always print in the same useful order."** Set order is not guaranteed for program logic.
4. **"A dictionary is unordered."** Modern Python guarantees dictionary insertion order, but dictionaries are still mappings, not sequences.
5. **"`bool('False')` is false."** It is true because the string is non-empty.
6. **"`None`, `0`, and `False` all mean the same thing."** They are distinct values with distinct types and meanings.
7. **"A tuple is always hashable."** Every tuple element must be hashable for the tuple itself to be hashable.

Practice problems with solutions

Problem 1: classify values

Write a list containing an integer, a float, a string, a Boolean, a list, and `None`. Print each value's type name.

Solution

PYTHON EXAMPLE

```
values = [7, 2.5, "seven", False, [7], None]

for value in values:
    print(type(value).__name__)
```

The type names are `int`, `float`, `str`, `bool`, `list`, and `NoneType`.

Problem 2: explain the result

PYTHON EXAMPLE

```
text = "5"
number = 5
print(text == number)
```

Solution: It prints `False`. The values have different types and do not compare equal. Convert `text` with `int(text)` if the program should compare numeric values.

Problem 3: build a dictionary

Create a dictionary with a person's name and city. Use `.get()` to display a missing `"country"` key with a default of `"Unknown"`.

Solution

PYTHON EXAMPLE

```
person = {"name": "Mina", "city": "Pune"}
country = person.get("country", "Unknown")
print(country)
```

Output: `Unknown`. The lookup does not add `"country"` to the dictionary.

Problem 4: fix Boolean parsing

Why is `bool("no")` true? Write an expression that treats `"yes"` and `"y"` as true after ignoring letter case and surrounding spaces.

Solution

PYTHON EXAMPLE

```
answer = input("Continue? ").strip().lower()
continue_program = answer in {"yes", "y"}
```

`"no"` is a non-empty string, so `bool("no")` is true. The solution checks explicit accepted answers instead.

Problem 5: preserve order while removing duplicates

Use the worked algorithm with `["a", "b", "a", "c"]`. What is the result?

Solution: `["a", "b", "c"]`. The second `"a"` is already in `seen`, so it is not appended again.

Key terms

Term	Short definition
Data type	A category describing a value and its supported operations
`int`	Whole-number type
`float`	Approximate floating-point numeric type
`complex`	Number with real and imaginary parts
`bool`	Type for `True` and `False`
`str`	Immutable Unicode text sequence
Sequence	Ordered collection accessed by position
Mutable	Able to change in place
Immutable	Not able to change in place
Mapping	Key-to-value collection such as `dict`
Hashable	Suitable for set membership and dictionary keys
`None`	Python's value for absence or no result

Chapter mastery checklist

- ☐ I can distinguish numeric values from numeric-looking strings.
- ☐ I can describe the main built-in Python types.
- ☐ I can choose a list, tuple, set, or dictionary for a simple problem.
- ☐ I understand `range` starts at its first value and stops before its stop value.
- ☐ I can explain list aliasing and shallow copying.
- ☐ I know that dictionary keys and set members must be hashable.
- ☐ I can explain the difference between mutable and immutable objects.
- ☐ I use `is None` for a `None` check.
- ☐ I can convert valid input and explain why invalid conversion can fail.
- ☐ I can debug a basic `TypeError` by checking the value types.
- ☐ I can dry-run the duplicate-removal algorithm.

Next chapter

Chapter 4 introduces arithmetic, comparison, assignment, logical, bitwise, membership, and identity operators.